Can AI-written data checks be trusted next to expert checks?

Thesis plan and the pilot work behind it

Tomash Oosterbaan
MSc Artificial Intelligence, University of Groningen
Graduation project with Avanade and Rabobank
November 2026, draft

1 Summary

Organisations check incoming data with validation rules: simple tests such as "a loan amount must be greater than zero" or "total assets must equal liabilities plus equity". Experts spend a lot of time writing these rules. AI language models, the technology behind tools such as ChatGPT, can now write such rules in seconds. Nobody knows how often these AI-written rules are right, or whether they add anything to the rules that experts already wrote.

This thesis measures that. We use four public sets of rules written by expert committees and regulators. They cover three areas that matter to a bank: the financial reports of companies (in the United States and in Europe), mortgage applications, and a global register of companies. Each rule set comes with a way to tell whether a rule works, which we call an answer key. Open-source AI models write rules, and we test those rules in three ways: against the answer keys, against errors that we place in the data on purpose, and against real errors that we label together with professionals.

The thesis delivers three things: evidence on when AI-written rules can be trusted and when not; a method to select the few AI rules worth keeping; and a working tool that lets a data team test AI-written rules before adopting them. No bank data is used.

The topic was not chosen on paper. It grew out of two pilot studies and two short checks between September and October 2026. Section 4 tells that story: what the first topic was, why it was dropped, and how each study led to the next.

2 The problem in plain words

Data from a new source is rarely ready for use. Before a bank uses it in reports, models or decisions, a data team checks it. Long Tran described this work at Rabobank as technical validation, business rules, monitoring thresholds and manual investigation of exceptions [1]. Supervisors expect such controls: the Basel Committee's principles for risk data (BCBS 239) and the European Central Bank's 2024 guide on risk data aggregation ask banks for accurate data and effective controls [2, 3].

Most of these controls are validation rules. A rule is a statement that every record must satisfy. Some rules are simple: a date must be a valid date, a country code must come from the official list. Others need knowledge of the business: in a balance sheet, total assets must equal total liabilities plus equity; the end date of a reporting period cannot lie before its start date. When a record breaks a rule, the rule raises an alert, and a person looks at it.

Writing good rules takes time and expertise, and there are always more data sources than experts. This is why AI is attractive. An analyst can describe a data source to a language model and receive twenty rules within a minute. Some of those rules are useful. Others look reasonable but are wrong. A wrong rule is not harmless, and it can fail in two ways. It can be too loose and let real errors through. Or it can be too strict and raise hundreds of alerts on correct data, so that the team wastes its time or starts ignoring the alerts. Two further problems are specific to AI. Asked again the next day, the model may write a different rule. And a rule that nobody can explain is hard to defend to an auditor.

So the team faces a simple question: the AI suggested twenty rules; which of them, if any, should we adopt? Today there is no good way to answer it. This thesis tries to give that question a measurable answer. Table 1 explains the terms used in the rest of this plan.

Term	Meaning in this plan
Validation rule, check	A test that every record must pass, for example "assets = liabilities + equity".
Expert rule set	Rules written and approved by a committee or regulator, independent of this study.
Answer key, gold standard	Cases for which the correct outcome is known, so that a rule or method can be scored.
Error injection	Changing values in a copy of real data on purpose, while recording exactly what was changed. Because we know where the errors are, we can count which ones a rule finds.
Alert, false alarm	An alert is a record that a rule flags. A false alarm is an alert on data that is correct.
Review budget	How many alerts a team can investigate, for example one per report.
Baseline	A simple, cheap method without AI. AI is only worth its cost if it does better than this.
Language model (LLM)	An AI model trained on large amounts of text and computer code. It can read a description and write text or code.
Open-weight model	A language model whose files can be downloaded and run on one's own computer, without sending data to a company.
Contamination	The model may have seen a rule, or its answer, during training. A test on such a rule measures memory, not skill.

Table 1: Key terms.

3 What earlier research says

This section describes six lines of earlier work that the thesis builds on. For each line it says what was found and what is still missing.

Finding errors in data. Research on data errors is older than the current AI wave. Abedjan et al. ran several error detection tools on real datasets and found that no single tool finds all errors; combining tools finds more, but also produces more alerts that a person has to check [4]. Later benchmarks such as REIN compared many detection and cleaning methods on the same data [5]. Newer detectors learn from the data itself. CDC predicts each value from the other columns and flags values that do not fit [6]; LUCARIO and RIOLU learn the usual format of values, such as the pattern of a postcode, and flag values with an unusual format [7, 8]; Auto-Test learns which values belong together from a large collection of tables [9]; Raha combines many simple detectors and learns from a few rows labelled by a person [10]. Jung et al. showed that the way errors are placed in data matters for how well detectors do [13]. What is missing for our purpose: these studies score detectors on public tables without expert rules, so they cannot say what a method adds to the rules a team already has.

Rules found automatically. Another line of work finds rules in data automatically. Deequ, developed at Amazon, suggests checks from simple profiles of the data, such as "this column is never empty" [15]. Auto-Validate learns patterns for text columns and limits how many false alarms they may raise [16]. A warning comes from Martin et al.: when rules are discovered automatically from data, most of them are false; they hold in the sample by accident [17]. This warning applies to AI-written rules as well.

AI that writes data-quality rules. Since 2023, several studies let language models write data-quality rules. Alexander et al. asked models to write tests for a dataset and compared them with one test suite written by a single data scientist; they ran each setup five times and found that results

varied between runs [18]. LeDQeR, from 2026, lets a model generate rules for a data-quality tool and then removes poor ones with four filters; the rules are scored on tables in which the errors are labelled [19]. AuDQ, also from 2026, uses several cooperating AI agents to discover rules and asks people to rate them; its authors write that their datasets have no reference rule set to compare against [20]. Akella et al. generate rules and code with extra retrieved knowledge and show each rule on a readable card [21]. A master's thesis at TU Wien generated rules with GPT models and compared them with five to ten "golden rules" per dataset, chosen by domain experts; whether each golden rule was covered was judged by hand, and 71 to 100% were [22]. Commercial data-quality tools have started to offer this feature [23]. What is missing: every study measures quality against labelled tables, human ratings or a handful of reference rules. None compares AI rules with a large, independent rule set written by experts, using an official answer key, and none asks what the AI rules add to rules a team already has.

Choosing among generated checks. Generating many candidate checks is easy; choosing the few worth keeping is the hard part. SPADE, from the University of California, Berkeley, generates checks for AI applications and then selects a small set that still catches the known failures while keeping false failures under a limit [24]. LeDQeR's filters do something similar for data [19]. What is missing: these selection methods look at the candidate checks on their own. They do not take into account the expert rules that are already in place, so they may keep checks that only repeat what the existing rules already catch.

AI and financial reports. Several recent benchmarks test language models on company reports in the XBRL format explained in Section 4.3. FinTagging tests whether models can attach the correct label to each number [25]; FinAuditing tests whether they can find errors across related documents [26]; SEC-FinTables tests whether they can spot inconsistencies in financial tables [27]. Models do reasonably well at finding numbers and poorly at the arithmetic relations between them. In all these studies the model acts as a judge of the data, not as a writer of rules that a person can read and approve. Errors in XBRL reports have been studied since the early years of digital filing [28], and corrected reports have been used as a signal of misreporting [29].

Testing checks with planted errors, and testing AI code. In software engineering, a common way to test a test suite is to plant small faults on purpose and count how many the tests catch. This is called mutation testing. The same idea has been used for data pipelines and data checks [30, 31]. We use it to test both the expert rules and the AI rules. Two findings about AI-written code also shape the design. Models perform worse on specialised, less common programming languages, such as the rule language used for financial reports [32]. And models give different code on different runs [33]. Finally, models may have seen public rules during their training; recent benchmarks for AI-written code therefore split test items by the date they were published [34]. We do the same.

The gap. Taken together, we did not find a study that (1) scores AI-written rules against large, independent expert rule sets using their official answer keys, (2) measures what AI rules add beyond the expert rules a team already has, at a fixed review budget, and compares ways of selecting them, or (3) checks the results on real errors labelled by people with domain knowledge. These three points define this thesis.

4 How the topic developed: the pilot work

Figure 1 shows the route. Each study answered a question and raised the next one. One method runs through all of them: error injection. We take real data, change values in a copy on purpose and record exactly what we changed. Because we know where the errors are, we can measure which errors a control finds, which it misses and how many false alarms it raises. The full pilot report is available separately [35].

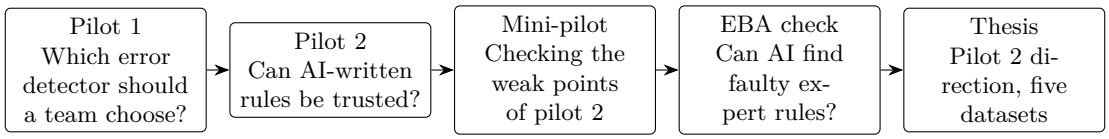


Figure 1: How the topic developed, September and October 2026.

4.1 Pilot 1: which error detector should a data team choose?

The first topic. My first thesis proposal was about choosing an error detector. Besides rules, a data team can use detectors: methods that look at the data and flag values that seem wrong, without anyone writing a rule. There are many of them (Section 3), and how well they work depends on the errors in the data. Jung et al. separate two properties of an error [13]. The type says what happens to a value, for example an amount multiplied by ten. The mechanism says which values are affected (Table 2).

Mechanism	Which values are affected	Example
ECAR	Completely at random	A random selection of monthly bills is multiplied by ten.
EAR	Depends on another field	The errors occur mainly in records from one source system.
ENAR	Depends on the value itself	Large bills are more likely to be multiplied by ten.

Table 2: Error mechanisms after Jung et al. [13]. The change is the same; only its placement differs.

Detectors are usually compared on errors placed completely at random. The idea was that a detector chosen that way might be the wrong choice for a bank, where errors often cluster in one source system or in large amounts. The thesis would compare the usual choice with a choice that takes the mechanism into account. The CURED demonstrator, which combines this error model with the CDC detector, was the starting point [14].

What we did. We used three public tables from the UCI repository: credit card clients (30,000 rows), the closest public equivalent of a bank credit file [36]; bank marketing (41,188 rows), with many category fields [37]; and white wine quality (4,898 rows), to see whether results hold outside finance [38]. For each table we used 800 rows for evaluation and up to 2,400 other rows as history, from which methods could learn what normal data looks like. We injected five types of errors: missing values, wrong data types, invalid codes, amounts multiplied by ten, and plausible substitutions, in which a valid value is replaced by another valid value. Each type was applied to 1, 5 or 10% of a column, under all three mechanisms, with five repetitions: 630 scenarios in total. A second round of 360 scenarios used errors that pass basic rules, such as a bill copied from the wrong month or an age that is one year off.

The comparison was always against what a team already has, not against nothing: rules for required fields, types and allowed codes; simple statistics learned from the history, such as unusual ranges and rare categories; and simple prediction models. On top of that we ran seven detectors (CDC, dBoost [11], LUCARIO, RIOLU, Auto-Test, Raha and a small experiment inspired by ZeroED, in which a language model helps label values [12]), and a very small local language model that was asked to propose rules. We measured how many injected errors each detector found that the existing checks missed, how many extra alerts it raised, and what was left when a person can review only 20 alerts per 800 rows. That last measure follows Abedjan et al. [4].

What we found. First, the rules caught every missing value, wrong type and invalid code; detectors only matter for errors that pass the rules. Second, the mechanism changed how much a detector added, but not which detector was best (Figure 2). For amounts multiplied by ten in the credit

table, CDC found 23% extra changed values when errors were placed at random, 14% when they clustered in one group and 27% when they hit large amounts. The other detectors added almost nothing under any mechanism. Third, the extra findings came at a high price. For errors that pass the rules, CDC found 11 to 21 of 40 changed values that the existing checks missed, but raised 520 to 900 extra alerts per batch of 800 rows. When a person can review only 20 alerts per batch, the extra alerts compete with the alerts of the existing checks. The net gain, the number of extra errors found within those 20 reviews, was 0.4 for the bank table, 0 for the credit table and -1.0 for wine. A negative number means that the detector's alerts pushed useful alerts of the existing checks out of the 20 reviews. A simple logistic regression, a basic statistical model, removed the small gain for the bank table. Auto-Test added 4 to 5 correct findings at 25 reviews on its own benchmark data, and none if the existing alerts were reviewed first. The small language model proposed four rules: two repeated existing checks and two repeated the column types. Where a copy of the original data existed, simply comparing the two found every change.

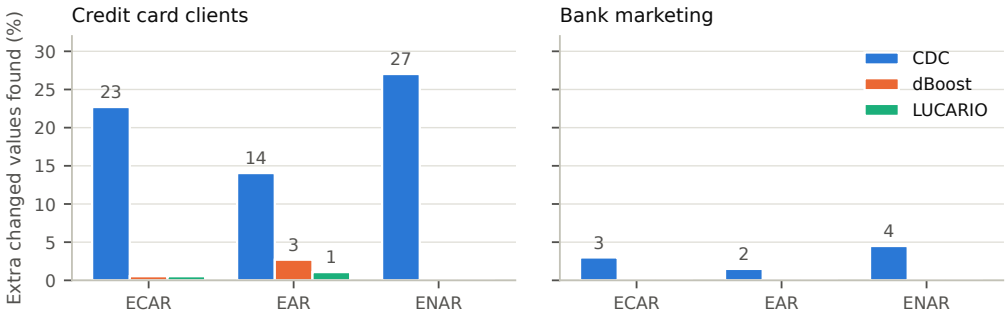


Figure 2: Pilot 1. Changed values found beyond the existing rules and statistics, for amounts multiplied by ten in 5% of a column, by mechanism. Mean of five runs. In the wine table no detector added anything.

4.2 Why we changed direction

The pilot was carried out carefully, with original code of the detectors, fixed settings and independent checks of the numbers, so the reasons to move on are about the question, not the execution.

1. The selection question did not hold. The mechanism changed the size of the effect, not the choice. With only one detector adding anything, there was nothing to select.
2. The remaining question, what an extra detector adds beyond existing checks when review time is limited, had already been studied [4, 5], and the Auto-Test authors present their method as a complement to existing rules [9]. The room for a new contribution was small.
3. All errors were made by us. The public tables have no record of real errors, so we could only show that a detector recovers our own changes, not that it finds real mistakes.
4. The public tables have no expert rules. We could not ask what AI adds beyond the rules that experts wrote, which is the situation of a real data team.
5. AI played only a minor role, which is a weak basis for a master's thesis in Artificial Intelligence.

Three lessons went into the next pilot: always compare with strong, cheap baselines; always report alerts next to findings; and use data that comes with expert rules and an answer key.

4.3 Pilot 2: can AI-written rules be trusted?

The idea. Instead of a detector that flags values, AI can write rules. For a bank this is attractive, because a rule can be read, approved and audited, which is much harder for a detector. Pilot 2 tested AI-written rules against a large public set of expert rules with an official answer key, and reused the injection method of pilot 1 as a stress test.

The data. American listed companies file their financial reports with the Securities and Exchange Commission (SEC) in a format called XBRL. In XBRL, every number is a separate fact with a standard label, a period, a unit and a precision, for example: `Revenues` = 5,000,000, for the year 2024, in US dollars, rounded to thousands. Because every company uses the same labels, computers can read and compare the reports. The Data Quality Committee (DQC) of XBRL US, a committee of preparers, software firms and auditors, publishes about 200 free rules that find common mistakes in these reports [39]. Rule DQC_0004, for example, checks that assets equal liabilities plus equity; its documentation gives the example message "Assets with a value of 340,000,000 is not equal to the total of Liabilities and Equity with a value of 350,000,000". Each rule has a written description, an official implementation in the rule language XULE that runs in the open-source program Arelle [40], and official test cases: example reports together with the exact messages the rule should give. These test cases are the answer key. Each rule also has a release date, so we can tell which rules a model could have seen during training. When a company finds a mistake, it can file a corrected report; comparing the two shows which numbers changed. We used the SEC Financial Statement Data Sets for 2023 to mid-2025: 68,875 reports, of which 1,294 were later corrected [41], and 240 XBRL reports for the experiments.

What we did. The existing controls were generic checks, three statistical checks learned from reports outside the sample (unusual sign, unusual size, unusual jump between periods), a deliberately naive rule that amounts are never negative, and the official DQC rules. We injected seven types of errors: a flipped sign, a value multiplied by 1,000, a changed amount, one changed digit, a wrong label, swapped periods and a removed fact. Two small open-source models (Qwen2.5-Coder-7B and Qwen3-8B) ran on a normal computer processor, without a graphics card. They wrote rules in three ways: from the description of a DQC rule, from a sample of a report, and from a list of error types that the existing controls missed. All AI code ran in a closed sandbox. As a comparison, a model also acted as a judge that reads a report and points at suspicious numbers. Before running anything we fixed an approval gate: an AI rule could only be adopted if it gave at most one alert per report that was never corrected, caught at least 20% of the errors the existing controls missed, and gave the same result when written again.

What we found. Reproducing expert rules: for rules released before the models, both models found about 41% of the official messages; for rules released after them, 3% and 13%. A rule translated by hand found 15 of 16 messages, so the test setup was not the cause. Writing the same rule three times often gave very different results, for two rules anywhere from 0% to 100%. Injected errors: the statistical checks caught two thirds of the errors multiplied by 1,000 and little else; the DQC rules caught 17 to 64% of most error types, but none of the removed facts. Together, the existing controls caught 27 of 83 injected errors. The best AI rule set caught 6 more, but gave on average 458 alerts per report that was never corrected, against 1.8 for the DQC rules (Figure 3). No AI rule set passed the gate. The alerts came from missing domain knowledge, not from programming mistakes: one rule compared the period type of each number, stored as "instant", with a list that codes it as "I"; another assumed that some amounts can never be negative, which is not true in XBRL. The AI judge caught 2 of 42 injected errors. Real corrections: of 20 corrected numbers that I checked against the documents, 9 were real errors in the original report, 3 were not, and 8 were unclear. Most corrections were accounting restatements rather than typing errors.

4.4 Mini-pilot: checking the weak points

Three short checks addressed the main doubts about pilot 2. First, none of the 66 test reports came from a company that was also used for development, so the results were not inflated by overlap. Second, we gave one model the official XULE code of nine rules it had failed. Three then worked; the other six still failed: three on carrying the logic over to the data, two on information that is only in the XBRL dictionary of labels (the taxonomy), and one on a programming error. Missing documentation therefore explains only a small part of the failures; missing domain knowledge explains more. Third, we judged the 24 single rules inside the AI rule sets separately instead of

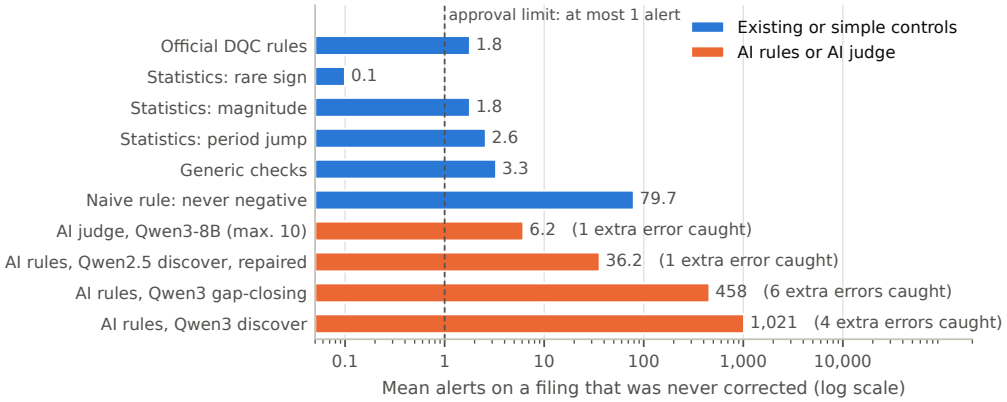


Figure 3: Pilot 2. Mean alerts per report that was never corrected, on a log scale (each step is ten times more). In brackets: injected errors caught beyond the existing controls. AI rule sets caught a few extra errors at a very high alert cost.

judging each set as a whole. We chose rules on half of the test reports and measured them on the other half. Without selection they caught 42 of the 195 errors the existing controls missed, at about 1,000 alerts per report. Keeping only rules that add something beyond the existing controls, within one alert per report, left one rule that caught 3 extra errors at 0.2 alerts per report. A generic filter that ignores the existing controls needed about twice as many alerts for the same gain. This selection step became the central idea of the thesis.

4.5 EBA check: can AI find faulty expert rules?

Before choosing, we tested a possible third direction. The European Banking Authority (EBA) decides what banks in the European Union report to their supervisors, for example about their capital and their financial position. With these reporting templates, the EBA publishes thousands of validation rules that every bank report must pass, such as "this total must equal the sum of these rows". Sometimes experts later find that a rule itself is wrong: it points at the wrong cell, uses the wrong arithmetic, or rejects reports that are correct. The EBA then switches the rule off ("deactivates" it) and has published lists of such incorrect rules since 2014 [42]. In the rule file of release 3.5, 1,694 of 20,562 rules carry a deactivation date. The question was: could an AI model, or a simple method, have spotted these faulty rules before the experts did?

How we tested it. The test had to be fair, so we used the release date of the AI model (Qwen3-8B, 29 April 2025) as a cutoff: the model cannot know about rules that were switched off after that date. We took the 5,537 hand-written rules that were still active on that date. Of these, 97 (1.75%) were switched off later; those count as faulty. Because the model needs about 30 seconds per rule, it scored a sample of 170 rules: 60 later switched off and 110 not. It saw each rule as it was first published, the tables it refers to, and up to eight other rules on the same table, and gave a score from 0 (certainly correct) to 10 (certainly wrong) with a one-sentence reason. Simple methods scored the same rules, for example by how many rules in the same table had been faulty before, by how unusual the formula was compared with the other rules on its table, by how recent the rule was, and simply by the length of the formula.

How to read the result. We used a measure called AUC. It answers one question: if you pick one faulty and one correct rule at random, how often does the method give the faulty rule the higher score? 50% is a coin flip; 100% is perfect.

What we found. The AI model scored 64% on the sample, better than a coin flip but worse than simply ranking rules by the length of their formula (71%). Within the same table, which is the practical question ("which of the rules on this table is wrong?"), the model scored 53%, close to a coin flip. For rules from the newest release, no method beat chance. The model's reasons were

mostly vague ("may be inconsistent"); of 24 faulty rules whose later correction is known, only one reason pointed at the part that was actually wrong. Methods that learn from earlier faulty rules pointed the wrong way: tables with problems in the past did not have problems later.

Why we did not choose it. There was no useful signal. The faulty rules also came in a few batches (23 tables, six dates), so there are few independent examples; some rules may have been switched off for technical reasons rather than being wrong; and the bank reports the rules are meant for are confidential, so the rules cannot be tried on real data. The check is kept as a short appendix with a negative result.

4.6 What we take from the pilot work

Four lessons hold across all studies. Cheap baselines are strong and must always be included. Alerts decide whether a method is useful, so they must always be counted next to findings. Injected errors are needed to measure anything, but real errors are needed to know whether it matters. And AI-written rules are unstable and noisy on their own, but selecting single rules against the existing controls separates useful from noisy rules. Three things change for the thesis: larger models on the university's computing cluster instead of two small models on a laptop; giving the model the domain knowledge it lacked; and more datasets, including real errors labelled with professionals. All choices that were made after seeing results in the pilots, such as alert limits, will be fixed in advance.

5 Research questions

Main question: under what conditions can validation rules written by open-weight language models reproduce expert rules and add reliable detection value beyond expert controls, at a review cost a data team can accept?

RQ1. Reproduce. Given the written description of an expert rule, how often does a model write a rule that gives the same results as the official one? How much does it help to give the model domain knowledge, and the chance to test its own rule on example data?

RQ2. Add and select. What do AI-written rules catch that the existing expert rules and simple statistics miss, at what alert cost? Which selection method keeps the useful rules: no selection, a generic filter, a SPADE-like selection, or a selection that takes the existing expert rules into account?

RQ3. Real errors and professional judgement. Do the selected rules also catch real errors, labelled by people with domain knowledge, and do professionals agree with the rules the method selects?

RQ1 asks whether AI can do what experts already did; RQ2 asks whether it can do something experts did not; RQ3 asks whether that holds outside our own experiments. Each question is answered on several datasets. If a result holds for company reports in the United States and in Europe, for mortgage data and for company register data, it is more likely to hold for a bank's own data. If it holds in one area only, that is also an answer.

From the pilots we expect that domain knowledge helps most for rules the models could not have seen during training; that the selection which takes existing rules into account gives more extra catches per alert than the other methods; and that real errors are harder to catch than injected ones. These are expectations to test, not claims.

6 Datasets

The thesis uses five datasets: four public datasets with expert rules, and a fifth that we build ourselves (Table 3). Each public dataset was chosen for four reasons: the data is public and free; the rules were written by experts independent of us; there is a way to score a rule; and the area matters to a bank. A bank lends to companies and reads their financial reports, sells mortgages, and must identify the companies it deals with.

Dataset	Rules and who wrote them	Answer key	Size (counted 31 October 2026)
D1. US company reports (SEC, XBRL)	DQC rules of the XBRL US Data Quality Committee [39]	Official test cases: example reports with the exact messages each rule should give	153 rules with test cases; 68,875 reports 2023 to mid-2025, of which 1,294 later corrected [41]
D2. European company reports (ESEF)	ESEF data quality rules of XBRL International, the international (IFRS) version of the DQC rules [43]	Official IFRS test cases; official expected results for real European reports [44]	25 rules, 22 with 64 test reports; expected results for 1,739 reports (from 2022); 27,626 reports in the public index [45]
D3. US mortgage applications (HMDA)	"Edits" of the US regulators FFIEC and CFPB, described in the Filing Instructions Guide [46]	The regulator's own open-source implementation with its unit tests [47]	318 edit codes in the implementation (237 validity, 72 quality, 9 syntax) and 330 test files; millions of public loan records per year
D4. Global company register (GLEIF)	207 data quality checks of the Global Legal Entity Identifier Foundation [48]	No public result per record; we build an answer key (Section 6.1)	207 checks in twelve quality criteria; public register of more than two million legal entities, updated daily
D5. Real errors (our own)	Not applicable	Corrected numbers in US reports, labelled by the student and professionals	Target: 300 corrected numbers, of which 100 labelled twice without seeing each other's labels

Table 3: Datasets. Numbers for D1 to D4 were counted by us in the official files and repositories.

D1, US company reports. This is the dataset of pilot 2 and the core of the thesis. It has the largest expert rule set with official test cases, and the pilot infrastructure already works on it.

D2, European company reports. Since the financial year 2020 (2021 in some member states), companies listed on a regulated market in the European Union must publish their annual reports in a European format based on XBRL, called ESEF. XBRL International publishes data quality rules for these reports, derived from the DQC rules but written for the international accounting standards (IFRS). They run in the same software. Besides test cases, there is an official file with the expected results of the rules on 1,739 real European reports. This dataset tests whether results from the United States also hold in Europe, with different accounting standards, different companies and reports in several languages. Because the expected results date from 2022, we also regenerate them by running the official rules ourselves.

D3, US mortgage applications. Under the Home Mortgage Disclosure Act (HMDA), mortgage lenders in the United States above certain size limits must report data on each mortgage application they receive. The regulators publish "edits", which are validation rules in plain language, in four kinds: syntax edits check the file format, validity edits check that each field contains an allowed value, quality edits flag values that are unusual and must be confirmed, and macro quality edits look at the file as a whole. Quality edit Q604, for example, checks that the reported county exists in the census list for that year. The regulator's own software that runs these edits is open source and comes with unit tests, which gives a reliable answer key. The data is tabular and close to the loan data a bank holds. The public data has been altered for privacy and has already passed the regulator's checks, so we test the edits on injected errors and only use edits that can be evaluated on the public fields.

D4, global company register. Companies and other legal entities that take part in financial transactions, for example trading in securities or derivatives, need a Legal Entity Identifier (LEI). GLEIF publishes the full register and 207 checks on it, written in a precise but not executable form: each check has a precondition, a condition and the expected outcome. Check C000020, for example, fails if a company's legal name is "n/a" or consists only of punctuation. This dataset tests rules written in yet another form, on reference data that every bank uses to identify its clients and counterparties. GLEIF publishes monthly summary reports, but not the result per record, so we build the answer key ourselves.

D5, real errors. This is the gold standard we build with professionals, described in the next section.

The four rule sets differ in form: the DQC and ESEF rules are code in a special rule language, the mortgage edits are plain-language descriptions with an official program behind them, and the GLEIF checks are semi-formal statements. This lets us see whether the results depend on how rules are written.

We considered and did not choose: the EBA rules for bank reporting (the bank reports are confidential, and our check found no signal), the public tables of pilot 1 (no expert rules, no real errors), and the research tables commonly used for error detection, which have the same limitation.

6.1 Gold standards we build ourselves

Real errors in company reports (D5). In pilot 2, corrected numbers turned out to be useful candidates but not yet a gold standard: of 20 checked, only 9 were clearly real errors, and two separate AI passes agreed on only half of them. We will therefore build a verified set. We draw 300 corrected numbers, spread over kinds of correction and over companies, and write a short labelling guide with worked examples. Each number gets one label: tagging error (wrong label or period), typing error, accounting restatement, not an error, or unclear. The student labels all 300 using the original documents. Two professionals with accounting or XBRL knowledge, recruited through Avanade, Rabobank or the university's Faculty of Economics and Business, each label the same 100 numbers without seeing other labels. We measure how well they agree with Cohen's kappa, a standard measure of agreement that corrects for agreement by chance [49], and discuss the disagreements. The data contains public company information only and will be published if the participants agree. Involving professionals requires approval of the faculty ethics committee, which we request in month one. We estimate about one day of work per professional.

An answer key for the GLEIF checks (D4). For 40 to 60 of the 207 checks we write small test records directly from the specification: one record that should pass, one that should fail and one where the check does not apply. We also write a reference implementation of these checks. A second person reviews both. As a sanity check we compare our failure rates on the real register with the figures in GLEIF's monthly reports.

Professional review of AI rules (RQ3). Three to five data-quality professionals each review 30 to 40 AI rules, with the evidence our tool produces, and decide whether they would adopt, reject or change each rule. We record each decision and the time it took, and compare the decisions with the automatic selection.

7 How we will answer the questions

Preparation. Before the main experiments we write down the data splits, alert budgets, metrics and approval criteria in a protocol and store it with a date. Data is split by company and by time, so that rules are never chosen and scored on the same reports. Rules are also split by publication date. A rule published in 2016 may be in the model's training data; a rule published after the model's release cannot be, so it is the fair test of whether the model can write a rule it has not seen.

RQ1, reproduce. For every rule with an answer key, each model writes the rule in three conditions.

(a) The model receives only the description of the rule. (b) The model also receives domain knowledge about the items the rule uses, for example whether an amount is normally positive, whether it is a balance on a date or a flow over a period, and which amounts add up to which total. (c) As in (b), and the model may run its rule on development data, look at the result and improve the rule, but never on the answer key itself. Each rule is generated five times in each condition. All generated code runs in a closed sandbox. We measure the share of the official messages a rule finds, the share of its own messages that are correct, whether the result is exactly right, and how much the five versions differ. Condition (b) tests the main finding of the mini-pilot, that missing domain knowledge causes most failures.

RQ2, add and select. We inject errors in reports and records that have no known errors: for example a flipped sign, a value multiplied by 1,000, one changed digit, a wrong label or category, swapped periods or a missing value. Where real corrections exist, the size of the injected errors follows them. The existing controls are the official expert rules, built-in format checks and simple statistical checks. The models then write rules aimed at the errors these controls miss. We compare four ways of selecting from those rules, each choosing on one half of the data and being scored on the other half:

1. no selection: keep every rule that runs;
2. a generic filter, as in LeDQeR: keep rules that run and are quiet on clean data;
3. a SPADE-like selection: keep a small set that balances catches and false alarms;
4. expert-aware selection: keep only rules that catch errors the existing rules miss, within a fixed alert budget, and that give the same result when written again.

The main measure is the number of extra errors caught at a budget of 0.5, 1 and 2 alerts per report or file. A simple example shows why the budget matters: a rule that catches ten extra errors but raises 450 alerts per report is useless to a team that can check one alert per report. As a second AI approach, a model also acts as a judge that reads a report and points at suspicious numbers directly.

RQ3, real errors and professionals. All controls and the selected AI rules run on the real-error gold standard (D5) and on reports that were never corrected. We measure the share of real errors flagged and the alerts on clean reports. We then compare the professionals' decisions with the automatic selection: does the method keep the rules that professionals would adopt?

Statistics. We report averages with 95% confidence intervals obtained by resampling over companies and rules, use paired comparisons where methods run on the same data, and report every run, including failed ones and results that go against our expectations.

8 Models

We only use open-weight models (Table 4). This has four advantages. No data leaves the organisation, which matters for a bank. There are no usage costs. The exact model file can be stored, so results can be reproduced later. And the release date is known, which we need to separate rules the model could have seen from rules it could not. Large models run on the university's computing cluster, Hábrók; the tool itself must also work with a small model on an ordinary computer, because that is what a data team would have.

Commercial models behind a paid online service are not used, because of cost and because a bank would not send its data to them without approval. Every AI result is compared with methods that use no AI.

Model	Released	Why
Qwen2.5-Coder 7B and 32B [50]	2024	Specialised in writing code; the 7B version was used in pilot 2, so results can be compared.
Qwen3 8B and 32B [51]	April 2025	General model that can reason step by step; the 8B version was used in the pilots.
gpt-oss 20B and 120B [52]	August 2025	A second model family, to check that conclusions do not depend on one company's models.
Newest open model at the start	To be fixed	Keeps the study current; fewer rules will be newer than this model, which we report.

Table 4: Models. "B" stands for billions of parameters, a rough measure of model size. Pilot 2 used models of 7 and 8 billion parameters.

9 Engineering artefact

Besides the research results, the thesis delivers a working piece of software, the engineering artefact. The pilots produced a command-line tool, `dqstress`. In the thesis it becomes a workbench that follows the steps of a data team that wants to use AI-written rules (Figure 4).

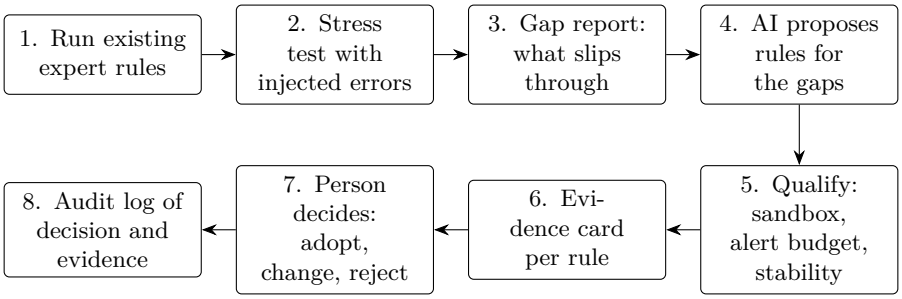


Figure 4: The workbench. The tool never adopts a rule by itself and never changes data.

An example shows how it would be used. A credit team wants to use the published financial reports of its business clients. It registers the data and the rules it already has (step 1). The tool injects known errors into a copy and reports which kinds of error slip through, for example flipped signs (steps 2 and 3). A local model proposes rules for those gaps (step 4). Each proposal runs in a sandbox and is scored on data it was not chosen on: how many of the missed errors it catches, how many alerts it raises on clean data, and whether it gives the same result when written again (step 5). The result is an evidence card per rule (step 6), on which a person decides (step 7), and the decision is stored with its evidence for an auditor (step 8). "The AI suggested twenty rules" becomes a short list of rules with evidence, of which perhaps none or a few are worth adopting.

The tool has an adapter for each data type in this study and one for ordinary tables, so a bank can run it internally on its own data and rules without sharing them. The language model sits behind a separate adapter, so an organisation can plug in the model it has approved. We evaluate the tool in three ways: every experiment in the thesis runs through it with one command, which makes the results reproducible; the professional review in RQ3 uses its evidence cards and measures how long a decision takes; and it is delivered with tests, documentation and an open-source licence.

10 What could be new

Much of this has been done before and we will not claim it: language models that write data-quality rules [19, 20, 21, 22], comparison with a small expert reference [22, 18], selection of generated checks under a false-alarm limit [24, 16], human approval of rules [20], injected errors to test checks [30, 31], and language models as judges of financial reports [26, 27].

What we did not find, and what this thesis would add:

1. A test of AI-written rules against four large, independent expert rule sets, using their official or verified answer keys, with rules split by publication date to rule out memorisation. Earlier work used labelled tables, human ratings or five to ten reference rules.
2. A measurement of what AI rules add beyond the expert rules a team already has, at a fixed review budget, and a comparison of selection methods that do and do not take those existing rules into account.
3. A verified public set of real errors in company reports, labelled by people with accounting knowledge, and an answer key for the GLEIF checks. Both can be reused by other researchers.

Running the same method on four datasets is not new in itself. The method stays the same on purpose, so that results can be compared, in the way one uses the same thermometer in different rooms. What is new is the evidence it produces: whether the answers hold across areas and countries. The contribution lies in the evaluation design, the selection method and the new gold standards, not in a new AI model. This field moves fast (LeDQeR and AuDQ appeared in September and October 2026), so the literature will be checked again every month and the claims adjusted if needed.

11 Planning and risks

Month	Work
1	Literature update; protocol fixed and stored; ethics request; access to the computing cluster; labelling guide; contact professionals.
2	RQ1 on D1 (US reports); GLEIF answer key built (D4).
3	RQ1 on D2, D3 and D4; start labelling real errors (D5).
4	RQ2 on D1: injected errors, AI rules for the gaps, four selection methods.
5	RQ2 on D2 and D3; professional labelling of D5 finished.
6	RQ3: real errors and professional review of AI rules; workbench completed.
7	Analysis across datasets; demonstration to Avanade and Rabobank.
8 to 9	Writing; buffer for delays.

Table 5: Nine-month plan.

D1 and D5 are the core. If time runs short, we cut in this order: the GLEIF dataset (D4) to RQ1 only, then the number of models, then the depth of RQ2 on the mortgage data (D3).

Risk	What we do about it
AI rules add little, even with larger models	The thesis is a measurement; a clear “no, and this is why” is a valid answer.
Few rules are newer than the models	Report confidence intervals; use rule sets with recent releases (DQC, HMDA).
Injected errors are not real errors	Base error sizes on real corrections; test on the real-error gold standard (D5).
Professionals have little time	Keep their part to about one day; the student labels everything, professionals label a shared subset.
Ethics approval takes long	Request it in month one; the public datasets do not depend on it.
Cluster access or queue delays	Start early; the small models from the pilots run on a laptop as a fallback.

Table 6: Main risks.

12 Value for Avanade and Rabobank

The thesis gives a data team three things it can use without sharing any data: evidence on what to expect from AI-written rules; a procedure to decide which ones to adopt; and a tool that produces the evidence an auditor would ask for. Because the tool runs locally with an open model and its own adapters, it can be tried on internal data and internal rules after the thesis, under the bank's own approval process.

References

[1] L. Tran, correspondence with the author, 4 September 2026 (unpublished; observations specific to his working context).

[2] Basel Committee on Banking Supervision, "Principles for effective risk data aggregation and risk reporting," Bank for International Settlements, January 2013.

[3] European Central Bank, "Guide on effective risk data aggregation and risk reporting," May 2024.

[4] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, and M. Stonebraker, "Detecting Data Errors: Where are we and what needs to be done?," Proceedings of the VLDB Endowment, vol. 9, no. 12, pp. 993–1004, 2016. doi:10.14778/2994509.2994518.

[5] M. Abdelaal, C. Hammacher, and H. Schoening, "REIN: A Comprehensive Benchmark Framework for Data Cleaning Methods in ML Pipelines," Proceedings of EDBT, pp. 499–511, 2023. arXiv:2302.04702.

[6] S. Jäger and F. Biessmann, "From Data Imputation to Data Cleaning: Automated Cleaning of Tabular Data Improves Downstream Predictive Performance," Proceedings of AISTATS, PMLR 238, pp. 3394–3402, 2024.

[7] Q. Qin, H. Li, and E. Merlo, "Unsupervised, Robust, and Lightweight Detection of Data Pattern Anomalies and Outliers," Information and Software Technology, vol. 192, article 108035, 2026. doi:10.1016/j.infsof.2026.108035.

[8] Q. Qin, H. Li, E. Merlo, and M. Lamothe, "Automated, Unsupervised, and Auto-Parameterized Inference of Data Patterns and Anomaly Detection," Proceedings of ICSE, pp. 2419–2431, 2025. doi:10.1109/ICSE55347.2025.00078.

[9] Q. Chen, Y. He, R. C.-W. Wong, W. Cui, S. Ge, H. Zhang, D. Zhang, and S. Chaudhuri, "Auto-Test: Learning Semantic-Domain Constraints for Unsupervised Error Detection in Tables," Proceedings of the ACM on Management of Data, vol. 3, no. 3, 2025. doi:10.1145/3725396.

[10] M. Mahdavi, Z. Abedjan, R. C. Fernandez, S. Madden, M. Ouzzani, M. Stonebraker, and N. Tang, "Raha: A Configuration-Free Error Detection System," Proceedings of SIGMOD, pp. 865–882, 2019. doi:10.1145/3299869.3324956.

[11] C. Pit-Claudel, Z. Mariet, R. Harding, and S. Madden, "Outlier Detection in Heterogeneous Datasets using Automatic Tuple Expansion," MIT CSAIL Technical Report MIT-CSAIL-TR-2016-002, 2016.

[12] W. Ni, K. Zhang, X. Miao, X. Zhao, Y. Wu, Y. Wang, and J. Yin, "ZeroED: Hybrid Zero-Shot Error Detection Through Large Language Model Reasoning," Proceedings of ICDE, pp. 3126–3139, 2025. doi:10.1109/ICDE65448.2025.00234.

[13] P. Jung, S. Jäger, N. Chandler, and F. Biessmann, "Towards Realistic Error Models for Tabular Data," ACM Journal of Data and Information Quality, vol. 17, no. 4, article 28, 2025. doi:10.1145/3774914.

[14] N. Chandler, S. Jäger, P. Jung, and F. Bießmann, "CURED: Creating, Understanding, and Repairing Errors Demonstrator," arXiv:2607.20140, 2026.

[15] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating Large-Scale Data Quality Verification," Proceedings of the VLDB Endowment, vol. 11, no. 12, pp. 1781–1794, 2018. doi:10.14778/3229863.3229867.

[16] J. Song and Y. He, "Auto-Validate: Unsupervised Data Validation Using Data-Domain Patterns Inferred from Data Lakes," Proceedings of SIGMOD, 2021.

[17] A. Martin, E. C. de Almeida, O. Romero, and A. Queralt, "How and Why False Denial Constraints are Discovered," Proceedings of the VLDB Endowment, vol. 18, no. 10, pp. 3477–3489, 2025. doi:10.14778/3748191.3748209.

[18] R. Alexander, L. Katz, C. Moore, M. W.-C. Wong, and Z. Schwartz, "Evaluating the Decency and Consistency of Data Validation Tests Generated by LLMs," arXiv:2310.01402, 2023.

[19] A.-C. Glock, T. Hütter, J. Fürnkranz, W. Wöß, A. Dominka-Kiss, and L. Ehrlinger, "Data Quality Rule Generation with LLMs," arXiv:2609.06053, 2026.

[20] J. Biggiogera, N. Colombo, K. Saxena, B. Sarmah, and S. Pasquali, "AuDQ: Auditable Data Quality through Agentic Rule Discovery," arXiv:2610.26031, 2026.

[21] A. Akella, A. Kaul, K. Narayanam, and S. Mehta, "Quality Assessment of Tabular Data using Large Language Models and Code Generation," Proceedings of EMNLP 2025: Industry Track, pp. 2713–2748, 2025. doi:10.18653/v1/2025.emnlp-industry.183.

[22] A. Schneider, "Template-Guided Rule Generation and Evaluation for Data Quality," MSc thesis, TU Wien, 2025. Supervisor G. Gottlob.

[23] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß, "Evaluating Data Quality Tools: Measurement Capabilities and LLM Integration," arXiv:2604.09163, 2026.
[24] S. Shankar, H. Li, P. Asawa, M. Hulsebos, Y. Lin, J. D. Zamfirescu-Pereira, H. Chase, W. Fu-Hinthorn, A. G. Parameswaran, and E. Wu, "SPADE: Synthesizing Data Quality Assertions for Large Language Model Pipelines," arXiv:2401.03038, 2024.
[25] Y. Wang, L. Qian, X. Peng, et al., "FinTagging: Benchmarking LLMs for Extracting and Structuring Financial Information," arXiv:2505.20650, 2025.
[26] Y. Wang, K. Wang, S. Yang, J. Patel, et al., "FinAuditing: A Financial Taxonomy-Structured Multi-Document Benchmark for Evaluating LLMs," arXiv:2510.08886, 2025.
[27] S. Ke, Q. Wu, H. Li, and L. Cao, "SEC-FinTables: Evaluating Large Language Models for Detecting Logical Inconsistencies on Tabular Data," Findings of ACL 2026, pp. 15587–15607, 2026. doi:10.18653/v1/2026.findings-acl.764.
[28] R. Debreceny, S. Farewell, M. Piechocki, C. Felden, and A. Gräning, "Does it add up? Early evidence on the data quality of XBRL filings to the SEC," Journal of Accounting and Public Policy, vol. 29, no. 3, pp. 296–306, 2010. doi:10.1016/j.jaccpubpol.2010.04.001.
[29] N. C. Brown, R. M. Crowley, and W. B. Elliott, "What Are You Saying? Using Topic to Detect Financial Misreporting," Journal of Accounting Research, vol. 58, no. 1, pp. 237–291, 2020. doi:10.1111/1475-679X.12294.
[30] H. Homayouni, S. Ghosh, and I. Ray, "An Approach for Testing the Extract-Transform-Load Process in Data Warehouse Systems," Proceedings of IDEAS, pp. 236–245, 2018. doi:10.1145/3216122.3216149.
[31] S. Schelter, T. Rukat, and F. Biessmann, "JENGA: A Framework to Study the Impact of Data Errors on the Predictions of Machine Learning Models," Proceedings of EDBT, pp. 529–534, 2021.
[32] S. Joel, J. J. Wu, and F. H. Fard, "A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages," ACM Transactions on Software Engineering and Methodology (accepted), arXiv:2410.03981.
[33] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation," ACM Transactions on Software Engineering and Methodology, vol. 34, no. 2, 2025. doi:10.1145/3697010.
[34] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," Proceedings of ICLR, 2025. arXiv:2403.07974.
[35] T. Oosterbaan, "Data-quality controls with and without AI: what two pilot studies showed," project repository `masterthesis_direction3_pilot`, October 2026.
[36] I.-C. Yeh, "Default of Credit Card Clients," UCI Machine Learning Repository, 2009. doi:10.24432/C55S3H.
[37] S. Moro, P. Rita, and P. Cortez, "Bank Marketing," UCI Machine Learning Repository, 2014. doi:10.24432/C5K306.
[38] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Wine Quality," UCI Machine Learning Repository, 2009. doi:10.24432/C56S3T.
[39] XBRL US Data Quality Committee, "DQC rules (dqc_us_rules)," rules, documentation and test suite, version 31. `https://github.com/DataQualityCommittee/dqc_us_rules`
[40] Arelle, open-source XBRL platform. `https://arelle.org`
[41] U.S. Securities and Exchange Commission, "Financial Statement Data Sets," quarters 2023q1 to 2025q2. `https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets`
[42] European Banking Authority, "Validation rules," rule workbooks including deactivation history. `https://www.eba.europa.eu/risk-and-data-analysis/reporting/reporting-frameworks/validation-rules-packages`
[43] XBRL International, "ESEF Data Quality Rules." `https://esef-dqr.xbrl.org`
[44] XBRL International, "ESEF DQR Conformance Suite." `https://gitlab.xbrl.org/xii/dqr-conformance`
[45] XBRL International, "filings.xbrl.org," public repository of ESEF filings. `https://filings.xbrl.org`
[46] Federal Financial Institutions Examination Council and Consumer Financial Protection Bureau, "Filing Instructions Guide for HMDA data collected in 2025." `https://ffiec.cfpb.gov/documentation/fig/2025/overview`
[47] Consumer Financial Protection Bureau, "HMDA Platform," open-source filing and validation software. `https://github.com/cfpb/hmda-platform`
[48] Global Legal Entity Identifier Foundation, "Data Quality Rule Setting," version 3.5, and monthly data quality reports. `https://www.gleif.org/en/lei-data/gleif-data-quality-management`
[49] J. Cohen, "A Coefficient of Agreement for Nominal Scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.
[50] B. Hui, J. Yang, Z. Cui, J. Yang, et al., "Qwen2.5-Coder Technical Report," arXiv:2409.12186, 2024.
[51] A. Yang, A. Li, B. Yang, B. Zhang, et al., "Qwen3 Technical Report," arXiv:2505.09388, 2025.
[52] OpenAI, "gpt-oss-120b & gpt-oss-20b Model Card," arXiv:2508.10925, 2025.